

FoodBridge — React Frontend Mentorship

Day 1 Notes — Project Setup, Tailwind CSS, Routing Basics

1. Project Initialization

Created the React frontend project separately from the Spring Boot backend, using Vite as the build tool.

```
npm create vite@latest foodbridge-frontend -- --template react
cd foodbridge-frontend
npm install
npm run dev
```

Key point: Frontend (React/Node) and backend (Spring Boot/Maven) are kept in separate sibling folders — never nested inside each other. They communicate only via HTTP/REST APIs, regardless of folder location.

2. Vite Project Structure (Default)

Understood the default Vite output:

```
foodbridge-frontend/
├── node_modules/  -> installed packages
├── public/        -> static files (favicon etc.)
├── src/
│   ├── App.jsx    -> root component
│   ├── main.jsx   -> entry point, renders App into index.html
│   └── index.css
├── index.html     -> has <div id="root"> that React renders into
├── package.json   -> dependencies & scripts
└── vite.config.js -> Vite configuration
```

3. Packages Installed

Package	Purpose
react-router-dom	Client-side routing (multiple pages, no full reload)
axios	HTTP client for calling Spring Boot REST APIs
react-toastify	Toast/success/error notifications
tailwindcss + @tailwindcss/vite	Utility-first CSS framework, integrated via Vite plugin

Note: No Redux used — React Context API is sufficient for storing auth state (user, role, JWT) at this project's scale.

4. Tailwind CSS Setup (v4)

Core concept: Instead of writing separate CSS rules, Tailwind uses small utility classes directly on elements. Example:

```
<h1 className="text-3xl font-bold text-blue-600">Hello</h1>
```

text-3xl = font size · font-bold = bold weight · text-blue-600 = blue shade (600 = intensity)

Installation:

```
npm install tailwindcss @tailwindcss/vite
```

vite.config.js:

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

src/index.css (entire content):

```
@import "tailwindcss";
```

Common mistake made & fixed: pasting the new vite.config.js content below the old one instead of replacing it, causing two 'export default' statements (invalid). Fix: delete the old block entirely.

5. Cleanup

Removed unnecessary boilerplate before building real features:

- Deleted src/App.css (no longer needed — Tailwind replaces component CSS files)
- Removed unused sample asset react.svg from src/assets
- Confirmed main.jsx still imports ./index.css (needed to load Tailwind)

6. Folder Architecture (Planned)

```
src/
  api/          -> Axios instance + API call functions (by feature)
  assets/       -> images, logos
  components/   -> reusable UI pieces (Navbar, Button, Loader, ProtectedRoute...)
  context/      -> AuthContext (user, JWT, role)
  pages/        -> page components, one folder per page
    Home/Home.jsx
    About/About.jsx
    Contact/Contact.jsx
    (donor/ngo/volunteer/admin subfolders to be added later)
  routes/       -> route definitions + protected route logic
  utils/        -> helper functions (token expiry check, formatters)
  App.jsx
  main.jsx
  index.css
```

Reasoning: pages/ organized by role mirrors the app's real domain. api/ keeps Axios calls out of components (components call functions, not raw URLs). context/ handles global auth state without needing Redux. routes/ keeps role-based access logic separate from page UI code.

Pattern used for pages: each page gets its own folder (e.g. pages/Home/Home.jsx) so it can later have its own CSS module or sub-components alongside it.

7. Routing Setup (React Router)

Concepts:

- **BrowserRouter** — wraps the app, enables URL-based navigation without full page reloads
- **Routes** — container that matches and renders exactly one Route
- **Route** `path="/" element={}` — maps a URL path to a component
- **Link** (replaces the anchor tag) — navigates instantly without a full page reload (core SPA behavior)

Final App.jsx for today:

```
import { BrowserRouter, Routes, Route } from 'react-router-dom'
import Navbar from './components/Navbar/Navbar'
import Home from './pages/Home/Home'
import About from './pages/About/About'
import Contact from './pages/Contact/Contact'

function App() {
  return (
    <BrowserRouter>
      <Navbar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/contact" element={<Contact />} />
      </Routes>
    </BrowserRouter>
  )
}

export default App
```

8. Navbar Component

First reusable component, placed in components/ (not pages/) since it's shared UI:

```
import { Link } from 'react-router-dom'

function Navbar() {
  return (
    <nav className="flex gap-6 p-4 bg-white shadow">
      <Link to="/" className="text-gray-700 hover:text-blue-600">Home</Link>
      <Link to="/about" className="text-gray-700 hover:text-blue-600">About</Link>
      <Link to="/contact" className="text-gray-700 hover:text-blue-600">Contact</Link>
    </nav>
  )
}

export default Navbar
```

Tailwind classes used: `flex` (row layout), `gap-6` (spacing), `p-4` (padding), `bg-white shadow` (white bg + subtle shadow), `hover:text-blue-600` (color change on hover).

9. Git Commit (End of Day 1)

```
git add .
git commit -m "feat: setup Vite+React project, Tailwind CSS, routing skeleton, and Navbar"
```

- Initialized React project with Vite
- Installed `react-router-dom`, `axios`, `react-toastify`
- Configured Tailwind CSS v4 with `@tailwindcss/vite` plugin
- Cleaned up boilerplate (`App.css`, unused assets)
- Created base folder architecture (`api`, `components`, `context`, `pages`, `routes`, `utils`)

- Added Home, About, Contact pages under pages/<PageName>/ pattern
- Set up React Router with BrowserRouter, Routes, Route
- Built reusable Navbar component with react-router-dom Link"

10. Next Steps (Day 2)

- Build Login and Register pages
- Set up AuthContext for storing JWT + user role
- Create Axios instance with base URL + auth interceptor
- Build ProtectedRoute component for role-based access
- Connect Login/Register forms to backend APIs